

React Fundamentals

What React is, why it exists, and how to get started

1 What is React?

A JavaScript library for building fast, component-based user interfaces — maintained by Meta.

- ☐ Builds UIs from small, reusable pieces
- ☐ Uses a Virtual DOM for fast updates
- ☐ One-way (unidirectional) data flow
- ☐ Huge ecosystem: Router, Query, Next.js

💡 React doesn't replace JavaScript — it's just a smarter way to organize it.

2 Setting Up a Project

Two common ways to start a new React app today.

```
terminal

# Fast, modern tooling
npm create vite@latest my-app -- --template react
cd my-app && npm install && npm run dev

# Classic option
npx create-react-app my-app
```

💡 Vite starts in milliseconds — prefer it over Create React App for new projects.

3 JSX — HTML in JS

JSX lets you write UI markup directly inside JavaScript.

```
App.jsx

function App() {
  const name = 'React';
  return (
    <h1>Hello, {name}!</h1>
  );
}
```

💡 Curly braces { } drop you back into plain JavaScript inside JSX.

4 Rendering to the Page

How React actually mounts your app into the HTML.

```
main.jsx

import { createRoot } from 'react-dom/client';

createRoot(document.getElementById('root'))
  .render(<App />);
```

💡 Only one root render call — everything else happens inside components.



"Learn the shape of React once — reuse it everywhere."

Components

Build · Reuse · Compose

1 Function Components

The modern way to create components in React.

```
Welcome.jsx

function Welcome() {
  return (
    <div>
      <h2>Hello, React!</h2>
    </div>
  );
}
```

💡 Function components are simple, reusable, and preferred with Hooks.

2 Imports & Exports

Split code into separate files and reuse components anywhere.

```
Button.jsx

export default function Button() {
  return <button>Click me</button>;
}

// App.jsx
import Button from './Button';
```

💡 Use export default for one main export per file.

3 Composing Components

Build bigger UIs by nesting smaller components inside each other.

- ☐ Keeps each piece small and focused
- ☐ Easier to test and debug
- ☐ Encourages a consistent UI
- ☐ Great for building a component library

💡 If a component does too much, it's usually time to split it.

4 Children Prop

Use children to pass JSX content inside a component, like a wrapper.

```
Box.jsx

function Box({ children }) {
  return <div className='box'>{children}</div>;
}

<Box><h3>Title inside the box</h3></Box>
```

💡 children lets you nest elements inside any component, like HTML tags.

Props & State

Passing data in, and managing data that changes

1 Props — Passing Data

Props (short for properties) let you pass data into components.

```
Greeting.jsx

function Greeting({ name, age }) {
  return <h2>Hello, {name}! You are {age}</h2>;
}

<Greeting name='Alice' age={25} />
```

💡 Props flow one-way: parent to child, never the other way.

2 Read-only Props

Props are read-only and can't be reassigned inside the component.

```
User.jsx

function User({ name }) {
  // ❌ name = 'Bob' -- not allowed
  const displayName = name || 'Guest';
  return <h2>{displayName}</h2>;
}
```

💡 Need to change a value? That's a sign you need state, not a prop.

3 State with useState

State is data a component owns and can update over time.

```
Counter.jsx

import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

💡 Calling setCount schedules a re-render with the new value.

4 Props vs State

The distinction that trips up almost every beginner.

- ☐ Props: passed IN from a parent, read-only
- ☐ State: owned BY the component, can change
- ☐ Changing state triggers a re-render
- ☐ Changing props happens from outside

💡 Rule of thumb: if it changes over time, it's probably state.

🚀 "Props flow down. State stays local."

Hooks Essentials

useEffect · useRef · useContext

1 useEffect Basics

Run side effects — like data fetching — after a component renders.

```
Profile.jsx

useEffect(() => {
  fetchUser(id).then(setUser);
}, [id]);

// runs again only when 'id' changes
```

💡 An empty [] dependency array means 'run once, on mount only.'

2 Cleanup Functions

Return a function from useEffect to clean up timers, subscriptions, etc.

```
Timer.jsx

useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);
}, []);
```

💡 Skipping cleanup is the #1 cause of memory leaks in React apps.

3 useRef

Store a mutable value or reference a DOM element — without re-rendering.

```
Input.jsx

const inputRef = useRef(null);

function focusInput() {
  inputRef.current.focus();
}


```

💡 Updating a ref does NOT trigger a re-render — unlike state.

4 useContext

Share data across many components without passing props manually at every level.

```
ThemeToggle.jsx

const ThemeContext = createContext('light');

function ThemeToggle() {
  const theme = useContext(ThemeContext);
  return <button>{theme} mode</button>;
}
```

💡 Great for theme, auth, or language settings shared app-wide.



"Hooks let function components do everything classes could — and more."

Events & Conditional UI

Handling clicks, forms, and showing the right thing at the right time

1 Handling Events

React events are named in camelCase and take a function, not a string.

```
LikeButton.jsx

function LikeButton() {
  function handleClick() {
    console.log('Liked!');
  }
  return <button onClick={handleClick}>Like</button>;
}
```

💡 Pass the function itself — `onClick={handleClick}`, not `handleClick()`.

2 Controlled Inputs

Bind an input's value to state so React is the single source of truth.

```
SearchBox.jsx

const [query, setQuery] = useState('');

<input
  value={query}
  onChange={(e) => setQuery(e.target.value)}
/>
```

💡 Every keystroke updates state, and state re-renders the input's value.

3 Conditional Rendering

Show or hide UI based on a condition — three common patterns.

```
Status.jsx

{isLoggedIn && <Dashboard />}

{isLoading ? <Spinner /> : <Content />}

if (!user) return <LoginPrompt />;
```

💡 Use `&&` for show/hide, ternaries for either/or, early return for guards.

4 Lists & Keys

Render arrays of data with `.map()`, and always add a stable key.

```
TodoList.jsx

{todos.map((todo) => (
  <li key={todo.id}>{todo.text}</li>
))}
```

💡 Never use the array index as a key if the list can reorder.



“Good UIs react to state. That's the whole idea.”

Quick Reference

The essentials, one more time before you build

1 Core Hooks Cheat Sheet

The four hooks you'll reach for in almost every component.

- ☐ `useState` — local component data
- ☐ `useEffect` — side effects & data fetching
- ☐ `useRef` — DOM refs & mutable values
- ☐ `useContext` — shared/global data

💡 Master these four before reaching for a state management library.

2 Common Mistakes

The bugs almost every React developer hits at least once.

- ☐ Mutating state directly instead of using `setState`
- ☐ Missing dependency array in `useEffect`
- ☐ Using array index as a list key
- ☐ Forgetting to clean up effects (timers, listeners)

💡 React's linter (`eslint-plugin-react-hooks`) catches most of these early.

3 Folder Structure

A simple, scalable structure for a growing app.

```
src/  
  components/  
  pages/  
  hooks/  
  context/  
  App.jsx
```

💡 Group by feature once the app grows past ~15-20 components.

4 What's Next?

Once these are second nature, here's where to go.

- ☐ React Router — multi-page navigation
- ☐ Data fetching libraries — React Query / SWR
- ☐ State management — Zustand / Redux Toolkit
- ☐ Testing — React Testing Library

💡 Build 2-3 small projects before adding more libraries to your stack.



"You don't need to know everything. You need to know enough to build."